

Technical Design: Multi-Source Candidate Data Transformer

1. Pipeline Architecture

The system executes in five strictly bounded, deterministic stages:

Ingest: Adapter layer streams raw sources (CSV, ATS JSON, PDF Resume, GitHub API) into lightweight `CandidateFragment` dicts. Uses `ijson` for O(1) memory JSON parsing and regex-based fuzzy key extraction to tolerate messy schemas.

Normalize: Pure, stateless transformers enforce domain boundaries (E.164 for phones, ISO-3166 for countries, strict date whitelists, taxonomy fuzzy-matching for skills). No silent data invention; failures return verbatim with a penalty flag.

Resolve & Merge: A Union-Find algorithm groups fragments by normalized email, phone, or explicit ID.

Score & Assemble: Conflict resolution applies source weights and corroboration bonuses. Generates a strict `CanonicalProfile` (Pydantic v2) with full provenance tracking.

Project & Validate: A runtime `ConfigurableProjector` reshapes the canonical record based on JSON config, enforcing a dynamic Pydantic Contradiction Guard to catch required-vs-omit conflicts.

2. Canonical Schema & Normalization Formats

- **Phones:** E.164 format via the `phonenumbers` library for unambiguous global deduplication.
- **Location:** `{city, region, country}` where country is ISO-3166 alpha-2.
- **Dates:** Strict `YYYY-MM` or `YYYY`. Rejects day-level precision and refuses to invent months for year-only inputs.
- **Skills:** Canonicalized against a curated taxonomy using `rapidfuzz` (`token_sort_ratio ≥ 87`).

3. Merge & Conflict-Resolution Policy

The merge engine is the core of the pipeline, designed around the philosophy that *"wrong-but-confident is worse than honestly-empty."* It operates in four distinct phases:

- **Entity Resolution (Cascading Union-Find):** Fragments are grouped into candidate entities using a cascading key strategy. It first attempts to union fragments by **normalized email**, then falls back to **E.164 phone**, and finally to **explicit raw IDs**. This ensures high-precision linking without relying on fuzzy name matching.
- **Scalar Resolution (Trust, Not Majority):** When sources provide conflicting values for a scalar field (e.g., `full_name` or `headline`), the system does not vote. Instead, `_resolve_scalar` sorts candidates by static source weight (ATS/CSV=0.90, GitHub=0.85, Resume=0.70). It keeps the highest-trust value, caps its confidence at `_CONFLICT_CAP` (0.40) to reflect the divergence, and writes every dropped value to an **append-only conflict ledger** with full provenance. If values agree post-normalization, they earn a corroboration bonus (+0.10 per source).
- **Chronological Deduplication (Sweep-Line Algorithm):** For array fields like `experience`, the system prevents double-counting in `years_experience` math. `_dedup_exp` identifies overlapping date ranges for the same role/company using a sweep-line algorithm. It merges the boundaries into a single continuous timeline while unioning the source provenance. Inverted dates (`end < start`) trigger a -0.15 penalty and nullify the end date.
- **Skills & Links Union:** Skills are merged via taxonomy-matched union; duplicate canonical skills boost confidence. Links undergo URL normalization (stripping schemes, `www.`, and trailing slashes) before equality checks to prevent semantic duplicates.

4. Runtime Configuration (Projection)

Accepts a JSON config to reshape output without code changes. Supports path resolution (`skills[:5].name`), normalization overrides, missing-value policies (`null / omit / error`), and provenance injection. Uses a dynamic Pydantic `ContradictionGuard` to catch required-vs-omit conflicts at runtime.

5. Edge Cases & Scope Boundaries

Edge Cases Handled:

- **US State vs. ISO Country Collisions:** "San Francisco, CA" resolves to US, not Canada. `normalize_country` checks a 50-state lookup for the exact "City, ST" shape before the generic `pycountry` pass, preventing silent data corruption.
- **Trust-Weighted Conflict Resolution:** As detailed in Section 3, divergences are capped and logged rather than silently overwritten or averaged.
- **Chronological Integrity & Overlaps:** Inverted dates nullify the end date with a penalty. Overlapping roles are merged via sweep-line logic to prevent double-counting in `years_experience` math.
- **Year-Only Date Imprecision:** Bare `YYYY` dates assume January 1st. We apply an explicit extra penalty during `years_experience` interval math to reflect this unstated assumption.
- **GitHub Isolation:** GitHub profiles lack emails/phones. They remain isolated singletons unless explicitly linked via `--github-map`, avoiding heuristic false merges.

Deliberately Descope:

- **Fuzzy Name Linking:** Avoided to prevent confident-but-wrong false merges. Requiring an explicit crosswalk is a constraint, not a gap.
- **Scale Limits:** Ingestion is O(1) via `ijson`, but the merge phase materializes fragments in RAM (O(n)). Fine for thousands, not millions.
- **Statelessness:** No cross-run persistence; every invocation is a one-shot batch.
- **Resume Parsing:** Regex-only. Complex layouts degrade to unstructured `raw_lines` rather than hallucinating structure via LLMs.
- **Concurrency:** Single-process, synchronous adapters. An honest throughput ceiling under time pressure.